

The Hardware/Software Interface & AArch64 Programmer's Model

Architectural state, registers, memory organization, and foundational instruction mechanics

Dr. J. Burns

Computer Systems Architecture — AArch64 Reference Platform

Topic 3: Module Topics

① Architectural State & The Hardware Contract

- Concrete definition of architectural state vs. microarchitectural structures.
- The AArch64 programmer's model and processor execution modes.

② AArch64 Register File & Naming Conventions

- The 31 general-purpose registers: 64-bit (x0–x30) vs. 32-bit (w0–w30).
- Special-purpose registers: Zero register (xzr/wzr), stack pointer (sp), program counter (pc).
- Standard ABI register usage: arguments, return values, caller-saved, and callee-saved registers.

③ Memory Organization & Endianness

- Byte-addressable 64-bit address space, memory alignment rules, and byte ordering.

④ Core Instruction Classes & Execution Mechanics

- Data movement, arithmetic/logical operations, NZCV condition flags, and condition codes.

PART 1

Architectural State & Execution Model

What Is Architectural State?

- **Definition:** The complete set of software-visible hardware storage locations required to represent a program's execution at any discrete point in time.
- If execution is interrupted (context switch, interrupt, exception), saving and restoring this exact state allows program resumption without alteration of behavior.
- **Core Components in AArch64:**
 - General-Purpose Registers (x0–x30).
 - Program Counter (pc) and Stack Pointer (sp).
 - Current Program Status Register / Condition Flags (pstate / NZCV).
 - System control and configuration registers (e.g., TTBR0_EL1, SCTLR_EL1 for exception/virtualization management).
 - Byte-addressed memory contents.

Architectural vs. Microarchitectural State

Architectural State (Visible Contract)

- Defined strictly by the ISA specification.
- Software and compilers directly read and write this state.
- Examples:
 - Registers x0–x30
 - Program counter (pc)
 - Architectural memory space
 - Flag bits (N, Z, C, V)

Microarchitectural State (Hidden)

- Internal hardware implementation details.
- Invisible to software correctness; exists solely for performance.
- Examples:
 - Pipeline register latches
 - L1/L2/L3 cache line contents
 - Branch Target Buffers (BTB)
 - Physical register rename tables

Architectural Boundary: Hardware may execute instructions out of order across hidden buffers, but must commit results sequentially to architectural state.

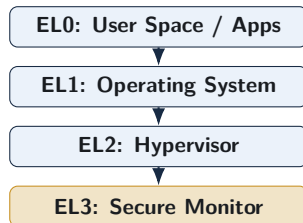
The AArch64 Execution State

- **AArch64:** 64-bit execution state introduced in ARMv8-A.
- **Flat 64-bit Virtual Addressing:** Registers hold 64-bit pointers (architectural address space up to 2^{64} bytes; OS implementations typically configure 48-bit or 52-bit spaces).
- **Uniform 32-bit Instructions:** Fixed 4-byte width, aligned to 4-byte memory boundaries.
- **Clean-Slate RISC Design (Eliminates A32 Quirks):**
 - No universal conditional execution on every instruction.
 - No load/store-multiple (replaced by paired `ldp/stp`).
 - Program Counter (`pc`) is decoupled from general-purpose registers.

Privilege Levels (Exception Levels)

AArch64 Exception Levels (EL0–EL3)

- **EL0 (Application):** Unprivileged user execution space (standard application code).
- **EL1 (OS Kernel):** Privileged operating system kernel (Linux, macOS/XNU).
- **EL2 (Hypervisor):** Virtualization layer managing guest OS instances.
- **EL3 (Secure Monitor):** Firmware, secure boot, and security management.



Course focus: Standard general-purpose user-space execution model (**EL0/EL1**).

AArch64 Register Architecture & ABI

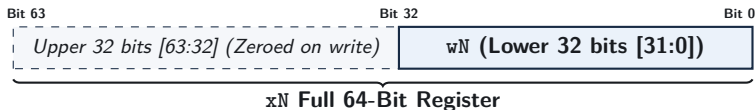
General-Purpose Registers: x0–x30

- AArch64 provides **31 general-purpose registers** (x0–x30), each **64 bits wide** (8 bytes).
- **Orthogonal Design:** Any register can serve as source, destination, or base address pointer.
- **Simplified ISA:** Eliminates dedicated register constraints from legacy architectures.



32-Bit Register Views: $w0$ – $w30$

- Every 64-bit x register has a corresponding 32-bit view denoted as $w0$ through $w30$.
- wN represents the **lower 32 bits (bits 31:0)** of register xN .
- Enables efficient execution of 32-bit integer operations without separate hardware register files.



Zero-Extension Rule on 32-Bit Writes

Writing to a w register **automatically clears (zero-extends)** the upper 32 bits of the corresponding x register to zero.

The Zero Register: `xzr` and `wzr`

- Encoded as register number 31 in most instruction contexts.
- **Read Behavior:** Always returns the constant value 0 (64-bit 0 for `xzr`, 32-bit 0 for `wzr`).
- **Write Behavior:** Discards any value written to it (serves as a discard target).
- **Architectural Benefit:** Eliminates dedicated zero-loading instructions and comparison formats.

Common Idioms Using the Zero Register

Assembly Instruction	Equivalent Operation	Purpose
<code>mov x0, xzr</code>	$x_0 \leftarrow 0$	Clear register x_0 to zero
<code>subs xzr, x1, x2</code>	Evaluate $x_1 - x_2 \rightarrow \text{flags}$	Compare x_1 and x_2 (alias: <code>cmp x1, x2</code>)
<code>str xzr, [x1]</code>	$\text{Mem}[x_1] \leftarrow 0$	Zero out a 64-bit memory word

Special-Purpose Registers: sp, pc, lr

Stack Pointer (sp)

- Dedicated register holding current stack top address.
- Must maintain **16-byte alignment** when accessing memory in standard AArch64 ABI.
- Arithmetic on sp is restricted to explicit stack allocations/deallocations.

Program Counter (pc)

- Holds memory address of executing instruction.
- Advances by +4 bytes on sequential execution.
- **Cannot** be read or written directly as a general data register (no direct `mov x0, pc`), but can be accessed indirectly via PC-relative addressing (`adr, adrp`).
- Modified via branch and call instructions.

The Calling Convention: Caller- vs. Callee-Saved

Caller-Saved Registers (x0–x17)

- Used for temporary computations and arguments.
- The caller must save them to memory/stack before making a subroutine call if their values are needed later.
- The called function (callee) is free to overwrite them without restoring.

Callee-Saved Registers (x19–x28)

- Used to hold long-lived variables across calls.
- If a called function modifies any of these, it **must save the original value to the stack** and restore it before returning.
- Protects caller state across subroutines.

Design Trade-off

Balancing caller- and callee-saved registers minimizes unnecessary memory traffic during deeply nested function calls.

AArch64 Register File Overview

Register	Architectural Role	Preserved Across Calls? (ABI)
x0–x7	Function arguments & return values	No (Caller-saved / Scratch)
x8	Indirect result location (struct return pointer)	No (Caller-saved)
x9–x15	Temporary / Scratch registers	No (Caller-saved)
x16–x17	Intra-procedure-call temporary registers (IP0, IP1)	No (Caller-saved)
x18	Platform register (reserved for OS / TLS)	Platform-dependent
x19–x28	Callee-saved registers	Yes (Callee-saved)
x29 (fp)	Frame pointer (base of current stack frame)	Yes (Callee-saved)
x30 (lr)	Link register (subroutine return address)	No (Caller-saved)
sp	Dedicated stack pointer (16-byte aligned)	Yes (Preserved)
xzr	Dedicated zero register (constant 0, writes discarded)	N/A (Hardware constant)

Process State: pstate and Condition Flags

- The processor maintains an internal process state register (pstate).
- The upper 4 bits contain the **NZCV Arithmetic Condition Flags**:

Flag	Name	Condition Under Which Flag Is Set to 1
N	Negative	Result bit 63 (or bit 31) is 1 (negative signed value)
Z	Zero	Computed result is exactly zero
C	Carry	Operation generated an unsigned carry-out (or no borrow in sub)
V	Overflow	Signed arithmetic overflow occurred (sign bit is invalid)

Selective Flag Updates

Standard ALU instructions (add, sub, and) do **not** modify flags. Only instructions with an “s” suffix (adds, subs, ands) or comparison instructions (cmp) update NZCV.

PART 3

Memory Model, Alignment & Endianness

Memory Organization & Byte Addressing

- Memory is organized as a contiguous sequence of **8-bit bytes**.
- Every byte has a unique 64-bit memory address.
- Data operands span multiple consecutive byte addresses based on type size:

Data Type	Size (Bytes)	Size (Bits)	AArch64 Architecture Term
Byte	1	8-bit	Byte (b)
Halfword	2	16-bit	Halfword (h)
Word	4	32-bit	Word (w)
Doubleword	8	64-bit	Doubleword (x)
Quadword	16	128-bit	Quadword / Vector element (q)

Memory Alignment Rules

- An access to an n -byte operand is **naturally aligned** if its memory address is an exact multiple of n : $\text{Address} \pmod n = 0$.
- **Alignment Requirements:**
 - 32-bit Word access: Address must be a multiple of 4 ($\text{Address} \pmod 4 = 0$).
 - 64-bit Doubleword access: Address must be a multiple of 8 ($\text{Address} \pmod 8 = 0$).
 - Instruction fetch: Address must be a multiple of 4.

Aligned Access (Single Transaction)

64-bit Word [Bytes 0–7]

Unaligned Access (Crosses Boundary)

Word Part 1

Word Part 2

Unaligned memory accesses can cross cache-line boundaries, requiring multiple memory transactions and degrading performance.

Byte Ordering: Little-Endian vs. Big-Endian

Storage of 32-bit Word 0x0a0b0c0d at Base Address 0x1000: Demonstrating how multi-byte values are mapped into sequential byte-addressable memory locations.

Little-Endian (AArch64 Default)

- **Least Significant Byte (LSB)** stored at the **lowest** memory address.

Address	Stored Byte
0x1000	0x0d (LSB)
0x1001	0x0c
0x1002	0x0b
0x1003	0x0a (MSB)

Big-Endian (Network Order)

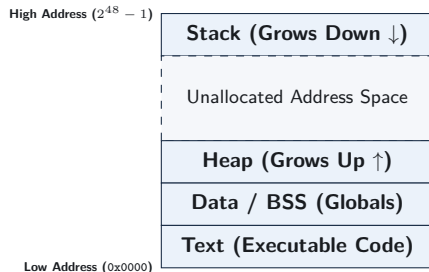
- **Most Significant Byte (MSB)** stored at the **lowest** memory address.

Address	Stored Byte
0x1000	0x0a (MSB)
0x1001	0x0b
0x1002	0x0c
0x1003	0x0d (LSB)

AArch64 is bi-endian by design, but standard OS environments (Linux, macOS, Windows) execute in **little-endian** mode.

The Process Address Space Layout

- **Text (Code):** Read-only machine instructions.
- **Data / BSS:** Global and static variables.
- **Heap:** Dynamically allocated memory via malloc; grows **upward** toward higher addresses.
- **Stack:** Local function variables and saved registers; grows **downward** toward lower addresses.



PART 4

Foundational Instruction Classes

Instruction Syntax Conventions

- Standard AArch64 assembly follows a strict operand ordering:

`mnemonic destination, source1, source2_or_immediate`

- **Destination First:** Leftmost operand receives the computed result.
- **Three-Operand Format:** Non-destructive; source registers are preserved.

Assembly Example	RTL Operation	Description
<code>add x0, x1, x2</code>	$x_0 \leftarrow x_1 + x_2$	64-bit register addition
<code>add w0, w1, w2</code>	$w_0 \leftarrow w_1 + w_2$	32-bit addition (clears upper 32 bits of x_0)
<code>add x0, x1, #16</code>	$x_0 \leftarrow x_1 + 16$	Addition with immediate constant

Arithmetic Instructions: Addition & Subtraction

Instruction	Operation	Flags Updated?
add xd, xn, xm	$x_d \leftarrow x_n + x_m$	No
adds xd, xn, xm	$x_d \leftarrow x_n + x_m$	Yes (Updates NZCV)
sub xd, xn, xm	$x_d \leftarrow x_n - x_m$	No
subs xd, xn, xm	$x_d \leftarrow x_n - x_m$	Yes (Updates NZCV)
neg xd, xm	$x_d \leftarrow 0 - x_m$ (Alias: sub xd, xzr, xm)	No
negs xd, xm	$x_d \leftarrow 0 - x_m$ (Alias: subs xd, xzr, xm)	Yes

Immediate Variants

- add xd, xn, #imm12: Adds a 12-bit unsigned immediate (0 to 4095).
- Can optionally be shifted left by 12 bits: add xd, xn, #imm12, lsl #12.

Logical Operations: Bitwise Manipulation

- Bitwise operations execute independently across all 64 (or 32) parallel bit slices.

Mnemonic	Operation	Bitwise Logic
<code>and xd, xn, xm</code>	Bitwise AND	$x_d \leftarrow x_n \wedge x_m$
<code>orr xd, xn, xm</code>	Bitwise OR	$x_d \leftarrow x_n \vee x_m$
<code>eor xd, xn, xm</code>	Bitwise XOR	$x_d \leftarrow x_n \oplus x_m$
<code>bic xd, xn, xm</code>	Bit Clear (AND NOT)	$x_d \leftarrow x_n \wedge (\sim x_m)$
<code>mvn xd, xm</code>	Bitwise NOT (Alias: <code>orn xd, xzr, xm</code>)	$x_d \leftarrow \sim x_m$
<code>ands xd, xn, xm</code>	Bitwise AND with Flags	$x_d \leftarrow x_n \wedge x_m \rightarrow \text{NZCV}$

Bit Clear (`bic`) Utility

`bic x0, x1, x2` clears every bit in x_1 where the corresponding bit in x_2 is set to 1.

Shift Operations: Barrel Shifter Integration

- AArch64 hardware includes a dedicated **barrel shifter** on the second operand path.
- Shift instructions exist as standalone operations or integrated directly into ALU instructions.

Mnemonic	Operation Name	Mathematical Meaning
<code>lsl xd, xn, #k</code>	Logical Shift Left	$x_d \leftarrow x_n \ll k$ (Multiplication by 2^k)
<code>lsr xd, xn, #k</code>	Logical Shift Right	$x_d \leftarrow x_n \gg k$ (Unsigned division by 2^k , zero-fill)
<code>asr xd, xn, #k</code>	Arithmetic Shift Right	$x_d \leftarrow x_n \ggg k$ (Signed division by 2^k , sign-fill)
<code>ror xd, xn, #k</code>	Rotate Right	Circular right bit rotation

Integrated Shifted Register Example

`add x0, x1, x2, lsl #3` $\implies x_0 \leftarrow x_1 + (x_2 \times 8)$ encoded as a **single instruction with hardware-shifted operand support**.

Comparison and Testing Instructions

- Comparison instructions evaluate operands, update NZCV flags, and discard the computed value.

Instruction	Underlying Operation	Flags Updated
<code>cmp xn, xm</code>	<code>subs xzr, xn, xm</code>	Evaluates $x_n - x_m \rightarrow$ NZCV
<code>cmn xn, xm</code>	<code>adds xzr, xn, xm</code>	Evaluates $x_n + x_m \rightarrow$ NZCV (Compare Negative)
<code>tst xn, xm</code>	<code>ands xzr, xn, xm</code>	Evaluates $x_n \wedge x_m \rightarrow$ NZCV (Bit Test)

Flag Interpretation Rule

- If $x_n == x_m$, `cmp xn, xm` sets the **Z (Zero)** flag ($x_n - x_m = 0$).
- If $x_n < x_m$ (signed), `cmp xn, xm` sets the condition flags such that $N \neq V$.

Loading Immediate Constants: movz, movk, movn

- In a fixed 32-bit instruction, an arbitrary 64-bit constant cannot fit in a single instruction word.
- AArch64 constructs large constants in 16-bit chunks using **wide-immediate instructions**:

Instruction	Operation Name	Semantic Action
movz xd, imm, lsl s	Move Wide with Zero	Loads 16-bit constant at shift $s \in \{0, 16, 32, 48\}$; clears rest
movk xd, imm, lsl s	Move Wide with Keep	Overwrites 16 bits at shift s ; keeps other bits
movn xd, imm, lsl s	Move Wide with NOT	Loads bitwise inverted 16-bit constant

Constructing a 32-bit Value (0x1234abcd)

movz x0, #0xabcd, lsl #0	$\implies x_0 = 0x000000000000abcd$	(Lower 16 bits loaded)
movk x0, #0x1234, lsl #16	$\implies x_0 = 0x000000001234abcd$	(Bits 31:16 overwritten)

Memory Access: Basic Load/Store Architecture

- Memory is accessed exclusively through explicit load (`ldr`) and store (`str`) instructions.
- Register size specifier determines data transfer width:

Instruction	Register Type	Transfer Width	Action
<code>ldr xt, [xn]</code>	64-bit (<code>xt</code>)	8 Bytes (Doubleword)	$x_t \leftarrow \text{Mem}_8[x_n]$
<code>str xt, [xn]</code>	64-bit (<code>xt</code>)	8 Bytes (Doubleword)	$\text{Mem}_8[x_n] \leftarrow x_t$
<code>ldr wt, [xn]</code>	32-bit (<code>wt</code>)	4 Bytes (Word)	$w_t \leftarrow \text{Mem}_4[x_n]$ (Zero-extended to x_t)
<code>str wt, [xn]</code>	32-bit (<code>wt</code>)	4 Bytes (Word)	$\text{Mem}_4[x_n] \leftarrow w_t$
<code>ldrb wt, [xn]</code>	8-bit (<code>wt</code>)	1 Byte	$w_t \leftarrow \text{Mem}_1[x_n]$ (Zero-extended)
<code>strb wt, [xn]</code>	8-bit (<code>wt</code>)	1 Byte	$\text{Mem}_1[x_n] \leftarrow w_t[7 : 0]$

Base register (x_n) holds the 64-bit memory pointer enclosed in square brackets `[xn]`.

Addressing Modes: Base with Immediate Offset

Mechanism ($EA = x_n + \text{offset}$)

- Instruction: `ldr xt, [xn, #offset]`
- Accesses memory at $x_n + \text{offset}$.
- **No side effects:** Base register x_n remains unchanged.

Code Examples

<code>ldr x1, [x0]</code>	Load $[x_0]$
<code>ldr x2, [x0, #8]</code>	Load $[x_0 + 8]$
<code>str x2, [x0, #16]</code>	Store $[x_0 + 16]$

Hardware Offset Range

Standard immediate offsets are unsigned 12-bit scaled values (for 64-bit doublewords, offsets up to +32760 bytes).

Advanced Addressing: Pre- and Post-Indexed

Pre-Indexed (Update Before Access)

```
ldr xt, [xn, #imm]!
```

- 1 Update base: $x_n \leftarrow x_n + \text{imm}$.
- 2 Access memory at updated address:
 $x_t \leftarrow \text{Mem}[x_n]$.

Note the exclamation mark (!) syntax.

Post-Indexed (Update After Access)

```
ldr xt, [xn], #imm
```

- 1 Access memory at current address:
 $x_t \leftarrow \text{Mem}[x_n]$.
- 2 Update base: $x_n \leftarrow x_n + \text{imm}$.

Base update happens *after* the load.

Efficiency Impact: Eliminates separate add instructions when traversing arrays or pushing/popping stack elements.

Loading and Storing Pairs (ldp and stp)

- **Core Purpose:** Transfer two adjacent 64-bit registers in a single instruction.
- **Primary Use Case (Stack Management):**
 - Combines two register movements into one atomic operation.
 - Efficiently saves and restores function linkage (Frame Pointer x_{29} and Link Register x_{30}) during function calls.
- **Performance Impact:** Issues a single 128-bit memory bus transaction, doubling data throughput compared to individual loads or stores.

Control Flow: Unconditional Branches

- Control flow instructions redirect program execution by modifying the Program Counter (pc).

Instruction	Name / Syntax	Operational Behavior
b label	Direct Branch	$pc \leftarrow \text{label}$ (PC-relative range: ± 128 MB)
br xn	Indirect Branch	$pc \leftarrow x_n$ (Jump to address held in register x_n)
bl label	Branch with Link	$x_{30} \leftarrow pc + 4$; $pc \leftarrow \text{label}$ (Subroutine call)
blr xn	Indirect Call	$x_{30} \leftarrow pc + 4$; $pc \leftarrow x_n$ (Function pointer call)
ret	Subroutine Return	$pc \leftarrow x_{30}$ (Default return via Link Register)

PC-Relative Addressing

Direct branch offsets are encoded as signed displacements relative to the current pc, making compiled code position-independent.

Conditional Branches & Condition Codes

Syntax: `b.cond label`

Executes branch to `label` only if the specified NZCV condition test evaluates to true.

Condition Code	Meaning	Flag Condition	Comparison Context
<code>b.eq</code>	Equal / Zero	$Z == 1$	Signed / Unsigned
<code>b.ne</code>	Not Equal / Non-Zero	$Z == 0$	Signed / Unsigned
<code>b.lt</code>	Signed Less Than	$N \neq V$	Signed
<code>b.le</code>	Signed Less Than or Equal	$(Z == 1) \vee (N \neq V)$	Signed
<code>b.gt</code>	Signed Greater Than	$(Z == 0) \wedge (N == V)$	Signed
<code>b.ge</code>	Signed Greater or Equal	$N == V$	Signed
<code>b.lo</code>	Unsigned Lower ($<$)	$C == 0$	Unsigned
<code>b.hi</code>	Unsigned Higher ($>$)	$(C == 1) \wedge (Z == 0)$	Unsigned

Control Flow Idiom: If-Else Translation

High-Level C Code

```
if (a == b) {  
    c = c + 1;  
} else {  
    c = c - 1;  
}
```

AArch64 Assembly Mapping

```
// Assume: x0=a, x1=b, x2=c  
cmp x0, x1          // Compare a and b  
b.ne .Lelse         // If a != b, jump to else  
add x2, x2, #1      // Then-body: c = c + 1  
b .Lend             // Skip else-body  
.Lelse:  
sub x2, x2, #1      // Else-body: c = c - 1  
.Lend:
```

Assembly Pattern: High-level conditions invert their test in assembly to jump *around* the then-body.

Control Flow Idiom: While-Loop Translation

High-Level C Loop

```
// Compute sum of array
uint64_t i = 0;
while (i < n) {
    sum += arr[i];
    i++;
}
```

AArch64 Assembly Translation

```
// x0=arr, x1=n, x2=i, x3=sum
mov x2, xzr                // i = 0
mov x3, xzr                // sum = 0
.Lloop:
cmp x2, x1                 // Test i < n
b.ge .Ldone               // Exit if i >= n
ldr x4, [x0, x2, lsl #3]   // Load arr[i]
add x3, x3, x4             // sum += arr[i]
add x2, x2, #1             // i++
b .Lloop                  // Repeat loop
.Ldone:
```

Conditional Select Instructions: Eliminating Branches

- Branch instructions introduce pipeline branch misprediction risks.
- AArch64 provides **conditional select instructions** that choose between operands without branching:

Instruction	Semantic Operation	C Ternary Equivalent
<code>csel xd, xn, xm, cond</code>	$x_d \leftarrow \text{cond} ? x_n : x_m$	<code>d = cond ? n : m;</code>
<code>cinc xd, xn, cond</code>	$x_d \leftarrow \text{cond} ? (x_n + 1) : x_n$	<code>d = n + (cond ? 1 : 0);</code>
<code>cset xd, cond</code>	$x_d \leftarrow \text{cond} ? 1 : 0$	<code>d = (cond) ? 1 : 0;</code>

Example: Finding Maximum of Two Values ($\max(x_1, x_2)$)

```
cmp x1, x2          // Compare x1 and x2
csel x0, x1, x2, gt  // If x1 > x2, x0 = x1; else x0 = x2 (0 branch penalties!)
```

Review and Self-Test Questions

Topic 3: Comprehensive Summary

- **Architectural State:** Precise hardware-software contract defined by registers ($x0$ – $x30$), flags (NZCV), stack pointer (sp), program counter (pc), system control registers, and byte-addressed memory.
- **Register Structure:** 31 orthogonal 64-bit registers ($x0$ – $x30$) with 32-bit aliases ($w0$ – $w30$); writing to a w register automatically clears the upper 32 bits.
- **ABI Rules:** $x0$ – $x7$ handle arguments and returns (caller-saved); $x19$ – $x28$ hold local variables across calls (callee-saved).
- **Load/Store Operations:** Arithmetic occurs exclusively in registers; memory is accessed via `ldr/str` using offset, pre-indexed, post-indexed, or paired (`ldp/stp`) modes.
- **Control Flow:** Branching evaluates NZCV flags updated by `adds/subs/cmp`; conditional select (`cse1`) eliminates branch penalties for small conditionals.

Self-Test Questions: State & Register Mechanics

- ❶ **Architectural vs. Microarchitectural State:** Why are CPU branch prediction tables and physical register rename structures classified as microarchitectural state rather than architectural state?
- ❷ **32-bit Register Write Semantics:** Suppose register x5 initially contains `0xffffffff_ffffffff`. If the instruction `mov w5, #0x1234` is executed, what exact 64-bit value resides in x5?
- ❸ **Zero Register Application:** Why does AArch64 provide a dedicated zero register (`xzr/wzr`) instead of dedicating a general-purpose register (like x0) to hold zero?
- ❹ **Calling Convention Responsibility:** If function `foo()` calls function `bar()`, and `foo()` needs to preserve values across the call in registers x3 and x20, which function is responsible for saving each register to the stack?

Self-Test Questions: Memory & Instruction Mechanics

- ❶ **Memory Addressing Differences:** Explain the exact difference in execution behavior and final register state between:
 - `ldr x1, [x0, #16]`
 - `ldr x1, [x0, #16]!`
 - `ldr x1, [x0], #16`
- ❷ **Endianness Evaluation:** The 32-bit word `0xdeadbeef` is stored at memory address `0x2000` on a little-endian AArch64 system. What is the exact byte value stored at address `0x2000`? At address `0x2003`?
- ❸ **Branch Elimination via `csel`:** Convert the following C code snippet into an equivalent branchless AArch64 assembly sequence using `cmp` and `csel`:


```
int64_t result = (val1 <= val2) ? (val1 + 10) : (val2 - 5);
```
- ❹ **Condition Flag Verification:** If `x0` contains 5 and `x1` contains 10, describe which NZCV condition flags are asserted after executing `cmp x0, x1`.

Looking Ahead to Topic 4: Assembly Programming

Topic 4 Preview (Arithmetic, Logic, & Control Flow):

- **ALU & Barrel Shifting:** Mastering arithmetic operations and free zero-penalty second-operand shifts.
- **Bitwise Logic & Field Extraction:** Efficiently manipulating bits with masks, bit clears, and `ubfx`.
- **Condition Flags & Comparisons:** Leveraging NZCV bits and choosing between signed vs. unsigned branch conditions.
- **Control Flow & Branchless Selection:** Translating loops and `if-else` blocks, and using `cse1` to eliminate branch penalties.